Metadata-Augmented Recursive Decomposition for Cost-Efficient Comprehension of Unbounded Documents

Parth Sangani[1, a, b, *] Arav Sharma[2, a, b] Anugrah Shetty[3, a, b] Tanish Sharma[4, a, b] Soni Sweta[5, a, b]

[a]Department of Computer Engineering

[b]Mukesh Patel School of Technology Management & Engineering, SVKM's Narsee Monjee Institute of Management Studies (NMIMS) Deemed-to-University, Mumbai-400056, India

[1]sanganiparth22@gmail.com

[2]arav.sharma0405@gmail.com

[3]anuadz2358@gmail.com

[4]tanishsharma619@gmail.com

[5]soni.sweta16@gmail.com

Corresponding Author[*]

***Abstract*—Recursive Language Models (RLMs) process inputs far beyond a model's native context window by holding the document in a REPL environment and letting a root model explore it programmatically, delegating reading to sub-model calls. We observe that this exploration is structurally blind: each recursive call receives a raw text slice with no view of the document's organisation, no record of what sibling calls have already established, and no statement of why it was invoked. On documents whose structure is exploitable—textbooks, manuals, standards—this discards precisely the information that makes the document navigable. We propose Metadata-Augmented Recursive Decomposition (MARD), an inference-time extension that threads a metadata envelope through every recursive call. The envelope carries a structural skeleton derived from the document's own text, findings accumulated from prior calls, and a parent-issued directive, and it grows as exploration proceeds. Exploration thereby becomes an act of confirming a hypothesis rather than of discovering blindly. We instantiate MARD on dependency-ordered study-material generation over textbook-scale documents, where a frontier "Scout" reads a small fraction of the document to emit a typed Master Plan and a fleet of cheap "Swarm" builders generate material that is joined in dependency order rather than book order. We evaluate MARD against vanilla RLM under matched models, matched recursion depth and identical structural controls, measuring output quality and tokens consumed independently, with three repeated runs per condition and a flat-context negative control. We additionally instrument the *groundedness* of generated content, after observing the baseline silently produce explanations without consulting the source document while reporting success.**

***Index Terms*—long-context language models, recursive inference, document structure, prerequisite ordering, inference-time methods, cost-efficient inference**

1. Introduction

The dominant approach to reasoning over documents that exceed a language model's context window is to reduce the document: chunk and retrieve, compress the prompt, or compact the conversation. Each discards information and each does so before the model has had a chance to decide what matters.

Recursive Language Models [1] invert this. The document is never placed in the prompt at all. It is bound to a variable inside a Python REPL, and the root model receives only metadata about it—its type, its length. The root then writes code to explore it: slicing, regular expressions, loops. Inside that REPL it may call `llm_query`, handing an extracted slice to a sub-model that reads it and returns an answer. The root is an orchestrator that decides what to read and delegates the reading. Because the root sees only truncated program output, its own context grows slowly, and inputs of several million tokens become tractable against a context window two orders of magnitude smaller.

1.1. The defect: recursion without orientation

We observe a specific deficiency in this design. Every sub-call is given a raw slice of text and nothing else. It does not know which document the slice comes from, where in that document it sits, what other sub-calls have already established, or why it in particular was invoked. It is, in effect, a reader handed three photocopied pages with no cover and no page numbers.

Two costs follow. First, orientation is re-derived on every call rather than established once and reused. Second, sub-calls cannot build on one another: a finding established in Chapter 3 is unavailable to the call that reads Chapter 9, even though the document itself may explicitly connect them. On a flat context—a shuffled transcript, a log file—neither cost is avoidable, because there is no structure to be oriented against.

But on a document with exploitable structure, the information being discarded is exactly the information the author supplied to make the text navigable: a table of contents, numbered sections, and in-text cross-references of the form "as we saw in Chapter 3."

1.2. Contribution

We introduce a *metadata envelope*: a compact, growing structure carried into every recursive call, comprising (i) a structural skeleton of the document, (ii) findings accumulated across prior calls, and (iii) a directive issued by the calling context stating what this call is for. Our claim, stated in the form we commit to before reporting any measurement:

> MARD's growing metadata envelope makes recursive document exploration structure-aware—improving output quality and/or reducing tokens consumed relative to vanilla RLM on documents with exploitable structure, while degrading gracefully to vanilla-RLM behaviour on documents without it.

Three properties of this sentence are deliberate. It credits the envelope, not recursion, because recursion is the base paper's. It is a disjunction over quality and tokens, because we do not know in advance which axis the effect appears on and measure both independently. And the graceful-degeneration clause is a prediction stated in advance, not a failure mode discovered afterwards; it is an objective of this work rather than a caveat on it.

A second property emerged from measurement rather than design, and we report it as a finding rather than folding it into the claim above. Because MARD's plan is a *typed contract* validated at the tier boundary, a builder receives its source span by construction rather than by a model matching text to a concept at generation time. We observed the vanilla baseline silently generate content without consulting the document at all, while reporting success (Section 4.3). The envelope's structural apparatus makes that failure mode loud rather than silent, which is a claim about *faithfulness and auditability* distinct from output quality.

1.3. What is not claimed

It is important to say this before a reader infers otherwise. The base paper's RLM already pairs a frontier root model with cheaper sub-model calls, and already recurses. Neither is our contribution. The base library that accompanies it already carries rich per-call metadata and even ships an example demonstrating metadata captured at every depth of the recursion tree. Section 3.1.1 draws the line precisely: that metadata flows upward and is observational, whereas MARD's envelope flows downward and is operative. Same word, opposite direction, and only one of the two changes what the model does.

We also make no claim to beat the state of the art on long-context benchmarks. Isolating MARD's contribution means comparing against vanilla RLM; the base paper's own results report a higher overall high-water mark from a different system, and that remains the standing bar [1].

1.4. Demonstrating application

We instantiate MARD on the generation of dependency-ordered study material from textbook-scale documents. This application is chosen because it makes the envelope's contribution measurable rather than anecdotal: a textbook declares its own prerequisite relations through in-text cross-references, so a generated ordering can be scored against ground truth extracted programmatically from the document itself, with no expert annotation and therefore no human-subjects component.

2. Literature Review

2.1. Recursive and long-context inference

Our work extends RLM [1] directly. A recent critique, SRLM [2], reports gains over RLM and argues that recursion may not be the primary driver of RLM's advantage, observing that within-window recursion sometimes degrades performance. We take no position on that dispute, and our design does not require one: the envelope reduces orientation cost, which is a claim about what each call is *given* rather than about what makes recursion work. The critique does, however, shape our methodology. If the objection to a reported improvement is that it may be sampling noise or prompt luck, the defence is not more documents but variance across repeated runs, a negative control, and a boundary condition stated in advance. Section 3.3 adopts all three.

Positional-degradation results [3] motivate the broader family of context-reduction approaches, of which prompt compression [4]–[6] and its recent survey [7] are representative. These are complementary rather than competing: they reduce what a call must read, whereas MARD changes what a call knows about what it reads.

2.2. Hierarchical decomposition of long documents

Hierarchical merging for summarisation [8] and multi-agent narrative summarisation [9] decompose long inputs into a tree and combine partial results upward. The structural difference from MARD is the direction of information flow: in merging schemes, context is assembled on the way up. In MARD, accumulated context is injected on the way down, before each unit of work executes.

2.3. Tiered and cost-aware inference

Routing and cascading approaches [10], [11] and hint-based delegation [12] allocate a frontier model selectively to control cost. MARD's Scout/Swarm split is an instance of the same economics, and we claim no novelty for it. What is ours is that the cheap tier receives a typed plan directive rather than an undifferentiated slice—an ablation we isolate directly (Section 4.4).

2.4. Prerequisite structure and educational applications

Concept prerequisite extraction has an established literature [13]–[15], as do LLM agents in education [16], [17]. We differ in the source of supervision: rather than learning prerequisite relations from labelled corpora, we treat each edge our system emits as a falsifiable prediction against the document's own declared cross-references.

2.5. Positioning against shipped systems

Two acknowledgements, which we state rather than let a reader supply. Ingestion capacity is not the gap—deployed products already accept large document collections—and at least one shipped product now generates sequenced study plans from uploaded course material. The line we draw is on the *source* of the sequencing: deployed systems that sequence do so diagnostically, ordering by what a learner has answered incorrectly, whereas our ordering is document-derived, extracted without supervision and scored against the text's own cross-references. We deliberately do not use any closed product as a quantitative baseline: their architectures are unpublished, they expose no token accounting, and they change without notice, so no such comparison would reproduce. No peer-reviewed source establishes what any particular shipped product's mind-map view does or does not encode, so we treat this as a qualitative observation supported by first-hand inspection rather than as a citable claim. Fig. 1 places three renderings of the same source document side by side. The distinction it is intended to make visible is not depth or resolution but *edge semantics*: in panels (a) and (b) an edge means *is contained in*, and in panel (c) an edge means *must precede*. **[VERIFY: Panels (a) and (b) are first-hand screenshots taken 27 Aug 2026 and are reproducible only in the sense that the procedure is stated; both products change without notice. Do not cite `tufino2025notebooklm` in support of this paragraph — it is a physics-education paper and does not address mind-map structure.]**

3. Methodology

3.1. Design

3.1.1. What the base library already provides: Formalising MARD requires first naming what it does not add, because the base library ships a great deal that sounds like our contribution. Its metadata types carry the root model, maximum depth, iteration count, backend and environment; its completion object carries the full execution trajectory—iterations, code blocks, and nested sub-calls, each with its own metadata, recursively; and per-call token and cost accounting is built in.

That metadata, however, flows *upward* and is observational: it is a record of what happened, assembled for inspection after the fact. It never re-enters a prompt and never influences a child call. This is checkable in the source rather than a matter of framing. When a parent spawns a child, the child is constructed with a fresh, empty logger—the library's own comment says the purpose is that the child's trajectory be captured, not that the parent's be communicated—and the child receives only the prompt string that the parent's REPL code passed to it. Nothing else crosses the boundary.

MARD's envelope flows *downward* and is operative. Skeleton, accumulated findings and parent directive enter the child's context before it runs, changing what the child can see and therefore what it produces.

One objection should be anticipated. The base library does expose a slot for injecting a small user-authored string alongside the root's context, and declines to populate it for children. The rebuttal is about what fills the slot, not whether a slot exists: that parameter is designed for a fixed, short, user-supplied string—its documentation's own example is the user's original question. The envelope is accumulated state that grows across calls. No mechanism in the base library constructs, propagates or compounds such state, and none carries one child's findings to the next. The library gives us somewhere to put an envelope; the envelope is ours.

3.1.2. The metadata envelope: Let D be a document and $S(D)$ a *skeleton*: an ordered structural index of D's chapters and sections with their positional spans, derived from D's own text. For a recursive call c we define the envelope

$$E(c) = \langle\, S(D),\ F(c),\ d(c) \,\rangle, \tag{1}$$

where $F(c)$ is the set of findings accumulated by calls ordered before c, and $d(c)$ is the directive issued by c's parent stating the purpose of the call. $E(c)$ is rendered into c's prompt in addition to its text slice. The envelope is bounded by construction: the per-call budget is approximately 300 tokens at chapter granularity, against a design ceiling of 500.[1] **[VERIFY: Confirm the 300-token figure against the rendered envelopes in the Pass 1 trace before this goes to the guide.]**

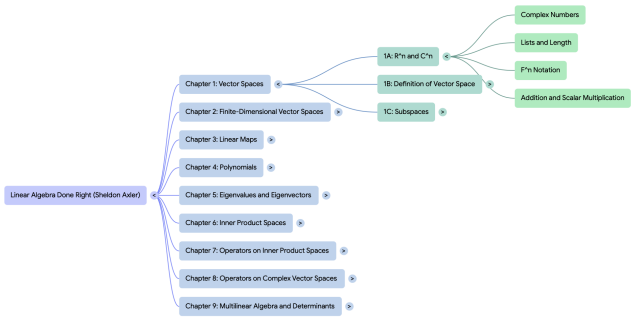
The design constraint that makes this non-trivial is that $E(c)$ must remain small while $F(c)$ grows monotonically with the number of completed calls. Findings are therefore summarised, not concatenated. The rendering additionally applies a recency window, so a call sees the most recent findings rather than all of them—which predicts that recovered cross-chapter relations will skew toward nearby chapters.

3.1.3. Pass structure: MARD proceeds in passes over D, all executed by the frontier tier.

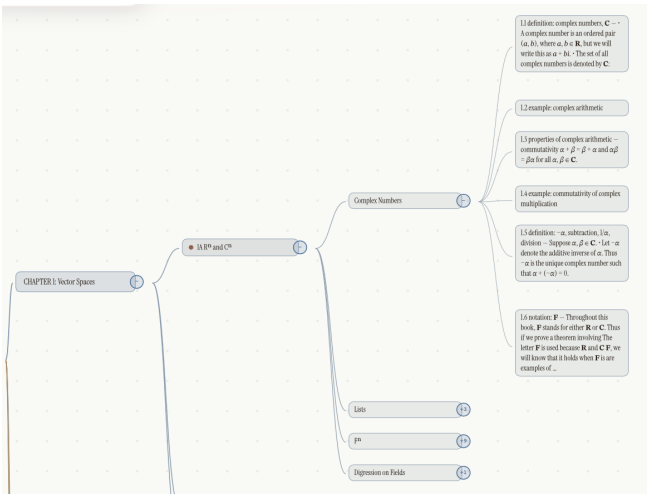
- **Pass 0 — skeleton extraction.** Derive $S(D)$ from the document's text. This pass is predominantly arithmetic over detected block structure rather than model inference; a single labelling call assigns topic labels. Crucially, the skeleton is text-derived only: where a source document ships a publisher-supplied bookmark outline, that outline is used solely as a yardstick for evaluating skeleton quality and is never an input (Section 3.3.3).
- **Pass 1 — enriched exploration.** One call per chapter, each carrying $E(c)$ and each contributing to F. The output is the Master Plan.

[1] Declaration of interest: panel (b) is produced by a prototype system built and maintained by one of the authors. It is included as an illustration of rendering granularity, not as an evaluated system, and no performance claim is made for it in this work.

[1] Section-level rendering measured an order of magnitude higher, which is why Pass 1 dispatches per chapter.



(a) A deployed notebook product's mind-map view. Edges denote containment; leaves are topic labels.



(b) The authors' prototype renderer,[1] one branch shown at full depth. Edges still denote containment, but the tree descends a level further, to typed atomic items retaining the source text and its notation.

[TODO: Panel (c): render MARD's concept graph for the same document from the compiled Master Plan. Edges are prerequisite assertions, each scorable against the document's own in-text cross-references. Blocked until Pass 1 emits a validated plan.]

(c) MARD's concept graph. Edges denote a prerequisite relation, not containment.

Fig. 1. Three renderings of *Linear Algebra Done Right*, 4th ed., the corpus document with the most explicit prerequisite structure and the target of the dependency-ordering case study rather than the primary measurement document. **All three were produced from an abridged version of the text, not the full book.** Panels (a) and (b) differ in resolution: both encode the containment hierarchy the document already declares through its headings, and (b)—shown for a single branch, since at whole-tree scale its leaves are illegible—descends a level further and preserves mathematical notation. Panel (c) differs in kind: its edges assert precedence rather than membership, and each is therefore a falsifiable prediction rather than a restatement of the table of contents.

- **Pass 2 — targeted deep dive.** Revisit spans that Pass 1 marked as uncertain, at greater depth.

This manuscript reports a two-pass configuration. The system evaluated here executes Pass 0 and Pass 1 only; Pass 2 is not run. This is stated here, in every results caption, and in the limitations, because describing a two-pass system as three-pass would be an overclaim. The decision was taken deliberately: Pass 1's implementation was completed one day before this project's feature freeze, and the reliability of a pass cannot be evidenced in a day. Layering a targeted deep dive on an unevidenced exploration pass would make a subsequent wrong result undiagnosable—it could not be attributed to either pass. The two-pass configuration is additionally, and by design, the depth-0 point of the depth sweep in Section 4.4, so this manuscript's number is a valid point on that curve rather than a one-off compromise.

3.1.4. The Master Plan as a typed contract: Pass 1 emits a *Master Plan*: a concept graph together with an ordered study sequence and the rationale for any reordering. It is validated against a schema at the tier boundary, and a malformed plan fails loudly rather than dispatching N subtly incorrect builders.

The field that carries the contribution is the concept graph's edge set. Each edge asserts that one concept is prerequisite to another, and each is therefore a falsifiable prediction scorable against the document's own in-text cross-references. Edges are validated at the boundary: an edge naming a concept that no call has declared is rejected and logged rather than silently admitted, which makes the rejection log itself a measurement of what exploration could and could not reach.

3.1.5. Two-tier execution and dependency-ordered joining: The Scout (frontier tier, single session) reads approximately 3–5% of D and emits the Master Plan. The Swarm (budget tier) then runs N builders in parallel, one per section, each receiving its section slice plus its plan directive. Because the builders fork and join, wall-clock time is $\max_i(\text{builder}_i)$ rather than $\sum_i$, and we report both so that the parallelism claim is checkable rather than asserted.

Builder outputs are joined in Master Plan order—the dependency-derived ordering—rather than in book order. We note explicitly that book order is itself an expert baseline, since the document's author already sequenced the material; reordering must therefore beat a real ordering, not a random

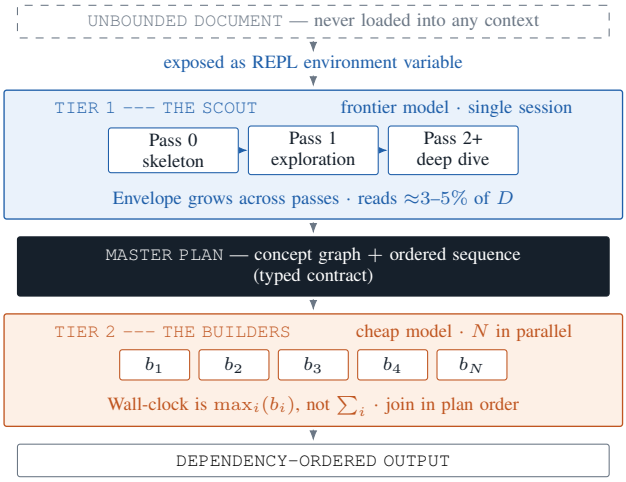


Fig. 2. MARD architecture. The document is never placed in a prompt; it is bound to a REPL variable and explored by the frontier tier, which emits a typed Master Plan that dispatches a parallel budget tier. Pass 2 is specified but *not executed* in the configuration evaluated here (Section 3.1.3); the system reported in this manuscript is two-pass.

one. The join additionally validates non-empty output per plan step and raises rather than returning a partial artefact, so a run that produced a short result fails visibly instead of scoring as a complete but poor one.

3.1.6. The stated precondition: MARD requires exploitable structure. On a flat context there is no skeleton to extract, F carries no positional meaning, and the envelope reduces to overhead. We therefore predict that MARD degenerates to vanilla-RLM behaviour on flat inputs, and we test this with a negative control rather than omitting it. This is an objective of the work, not an embarrassment to be discovered in review.

The control is a *structure-ablated variant of the primary document*: the same text with its section order shuffled and its heading markers removed. This is a deliberate choice over an external flat-context benchmark. It holds content constant and varies only structure, so it is a manipulation rather than a change of corpus, and it parallels the envelope ablation one level lower—A1 removes the accumulated structure the system builds, and this removes the structure the document supplies. The pipeline treats the resulting empty skeleton as a recorded condition rather than an error, so degeneration appears in the run log as a finding rather than as a missing value. An external flat-context benchmark subset is fixed and frozen in the project's evaluation artefacts and is run in the full study (Section 5).

3.2. Algorithm and Implementation

Model pair. The frontier tier (Scout) is `gpt-5.2`; the budget tier (Swarm, and the baseline's sub-calls) is `gpt-5-mini`. Rates were read from the provider's own pricing page on the day the rate card was written and are recorded once, in code, with their source and retrieval date; the accounting layer refuses to price a run against a rate more than thirty days old rather than let a stale figure ride.

We are explicit that this pair is selected on *role fit and cost*, not on a published benchmark comparison. The Scout runs once per document and must hold the skeleton and accumulated envelope across a sequence of chapter calls; the Swarm is N independent builders each consuming one section and a directive, where call volume rather than per-call ceiling dominates. The budget tier carries roughly 93% of the token volume at approximately one seventh the rate on both input and output. A model sweep that would test that reasoning is deferred to the full study, and we do not present the selection as a measured result.

Both arms are executed on a single implementation. Isolating the envelope requires that the two arms differ *only* in its presence. Executing the baseline on a separate codebase would confound the envelope with two independent implementations—differing prompt templates, REPL surfaces and stopping criteria—none of which is the variable under study. We therefore implement vanilla RLM as an envelope-disabled configuration of the same system, and treat the reference implementation accompanying the base paper [1] as the specification we implement against rather than as the artefact we measure.

The cost of this choice is that our baseline is our own implementation of RLM rather than the original authors'. We mitigate it in the only way available: by reproducing a published result from the base paper first-hand on our own harness before any number in Section 4 is trusted. That check is not a formality here—it is the sole evidence that the vanilla arm faithfully reproduces the published method. **[TODO: Report the reproduced base-paper figure and the target it was checked against. This check gates every number in Section 4.]**

Concurrency and sub-call size are bounded, identically for both arms. Runs were executed under a provider rate-limit tier that required bounding sub-call concurrency and sub-call input size. **[TODO: State the two values once fixed.]** Both bounds are applied to *both* arms and are recorded in every run's configuration snapshot. We report this because it is not a neutral implementation detail: throttling can degrade output completeness, and applying it asymmetrically would manufacture a difference between the arms that has nothing to do with the envelope.

The cost model is instantiated with measured token counts rather than a claimed multiple. Table I reports the projected cost of the full evaluation matrix from measured corpus quantities. We report the model rather than a headline ratio because provider rates move month to month, and a fixed multiple quoted against a rate that later disappears is precisely the kind of claim that decays into misinformation.

3.3. Experimental Setup

3.3.1. Corpus and licensing: Our corpus consists of openly licensed textbooks spanning distinct domains, under an open-license-only constraint on all source material. Table II summarises them. Licensing was verified against each publisher's own copyright page rather than a third-party aggregator.

TABLE I
COST OF THE EVALUATION MATRIX ON THE PRIMARY DOCUMENT. CORPUS QUANTITIES AND THE BASELINE'S PER-RUN COST ARE MEASURED; THE MARD FIGURE IS PROJECTED FROM THOSE QUANTITIES AT THE SAME RATES

Measured quantity	**Value**
Primary document, characters	2 468 431
Primary document, tokens†	≈617 000
Sections (Swarm builders)	120
Chapters explored by Pass 1	14
Published rates, per 1,000,000 tokens	
Frontier tier, input / output	$1.75 / $14.00
Budget tier, input / output	$0.25 / $2.00
Cost per run	
Vanilla RLM, measured‡	$0.32–$1.60
MARD, projected	≈$0.94
Remaining matrix, projected	≈$11

†Derived from character counts at an assumed ratio; pending confirmation with a tokeniser. ‡Three logged runs; the spread is reported rather than averaged, for the reason given in Section 4.2.

TABLE II
CORPUS DOCUMENTS AND LICENCES, VERIFIED AGAINST EACH SOURCE'S OWN COPYRIGHT PAGE

Document	**Licence**	**Role**
Introduction to Computer Science (OpenStax)	CC BY 4.0	Primary
University Physics, Vol. 1 (OpenStax)	CC BY-NC-SA 4.0	Secondary
Linear Algebra Done Right, 4th ed.	CC BY-NC 4.0	Secondary

OpenStax titles were selected specifically because they ship machine-readable learning objectives, glossaries and answer keys—the substrate required for document-native ground truth extracted without human annotation, and therefore without an ethics-approval process this project's timeline cannot absorb.

On the licence of the primary document. *Introduction to Computer Science* is CC BY 4.0 (plain attribution), confirmed against OpenStax's book-level licensing page. Individual chapter footers within the same book state CC BY-NC-SA; we treat the book-level page as authoritative and record the inconsistency rather than silently choosing one.

3.3.2. Ingestion and primary-document selection: Primary-document selection was made on measured evidence rather than convenience. Table III reports parse quality across the three candidates. No optical character recognition is used on any document: all three carry a text layer, and OCR would replace exact text with approximate text—decisively, OCR errors in glossary terms would corrupt the very reference set against which task quality is scored, silently and irreversibly.

University Physics is disqualified as primary on content loss, not formatting loss: its orphan-punctuation rate is nearly four times that of the primary document, and inspection confirms sentences from which equations have vanished entirely. That damage is not uniform across systems under test—a configuration that reads more of the document encounters more of it—and would manufacture a difference between MARD and vanilla RLM having nothing to do with either. *Linear Algebra Done Right* carries no learning objectives and no key-terms sections, so document-native ground truth cannot be extracted from it; it remains the strongest candidate for the dependency-ordering case study, which is scored separately.

TABLE III
PARSE QUALITY, PRODUCED BY THE INGESTION PIPELINE AND REPRODUCIBLE FROM THE CORPUS ARTEFACTS. ORPHAN PUNCTUATION COUNTS PUNCTUATION STRANDED WHERE A FORMULA FAILED TO EXTRACT

Measure	**introcs**	**physics1**	**axler**
Pages	939	959	404
Characters after cleaning	2 432 812	2 032 210	737 844
Headings detected	1 403	1 081	17
Learning-objective blocks	61	99	0
Key-terms blocks	14	17	0
Orphan punctuation (per 1k lines)	30.3	113.5	87.8
Warnings raised	0	1	3

Page mapping was verified by sampling 1200 blocks at random across the three documents and confirming each appears on the page it claims; 1199 passed, the single failure being a whitespace-normalisation difference in an index line rather than a mapping error.

3.3.3. Two structural confounds, and how they are controlled: Both concern structure reaching a model without that model having derived it. If structure arrives by another route, we would be reporting the publisher's work as MARD's.

1) **The publisher's bookmark outline.** All three documents ship one. The ingestion pipeline writes it to a separate artefact with an explicit provenance field and never merges it into the text stream, so that using it must be a deliberate decision rather than an accident. It serves as the yardstick for skeleton quality and is not an input to Pass 0.
2) **The printed table of contents inside the body text.** Subtler, because it arrives through the ordinary text channel: the primary document prints its full chapter list on page 7, so a model reading the opening pages obtains the skeleton for free. Front matter is detected, tagged, and excluded from the text stream supplied to every system.

Both controls are applied identically to the vanilla-RLM arm. If the two arms received different inputs, the comparison would not isolate the envelope.

3.3.4. Skeleton fidelity: Because the skeleton is the substrate for everything downstream, its quality is measured before it is used. Against the publisher outline, the primary document's text-derived skeleton achieves 82.2 % recall at 0.0 pages of boundary error. Boundary error of zero indicates that

where a section is recovered, its span is recovered exactly; the recall shortfall is missed sections, not misplaced ones.

3.3.5. Systems compared:

- **MARD** — two-pass, as specified above.
- **B1, vanilla RLM** — the isolation baseline: the base paper's architecture, a root REPL loop issuing flat sub-calls, on identical models, document and structural controls.
- **A1, envelope removed** — MARD's architecture with the envelope emptied. This is a *separate* run from B1 and serves a different purpose, discussed below.
- **Negative control** — MARD and B1 on the structure-ablated variant of the primary document, testing the graceful-degeneration half of the claim under content held constant.

B1 and A1 are deliberately distinguished, because they answer different questions. B1 compares MARD's architecture against the base paper's, and a win there changes several things at once—the envelope, the pass structure, the typed plan, two-tier dispatch and dependency-ordered joining. A1 holds the architecture fixed and removes only the envelope, and is therefore the only comparison that attributes an effect to the envelope specifically, which is what this paper's claim rests on. Reporting B1 alone would license the weaker conclusion that our pipeline is better, not that the envelope is why.

The full study additionally runs full-context, naive chunking and embedding-RAG baselines first-hand.

Recursion depth is held fixed. Both arms run at a matched recursion depth in which sub-calls are plain completions. This is the base paper's primary reported condition. Holding depth fixed is not incidental: if the arms differed in depth, the measured difference would confound the envelope with an extra layer of recursion, and the isolation claim would be void. **[VERIFY: The mapping between MARD's pass-depth numbering and the control library's recursion-depth parameter is off by one and is currently derived from project documentation rather than from the implementation's call structure. Confirm against the Pass 1 source before this paragraph is final.]**

3.3.6. Metrics: Every run of every system records seven fields, and a number that cannot be traced to all seven from a logged run is not admitted to this paper: task score; tokens consumed (input and output, summed over every model call and reported separately in the log); calls issued, split by tier; cost at the same-day published provider rate; wall-clock latency (both $\max$ and $\sum$ over builders); run identifier; and a full configuration snapshot including model identifiers, prompt-template versions, depth setting, active ablation, and document. Systems producing per-concept output additionally record the groundedness fields defined in Section 4.3.

Task score. The evaluated output modality is explanations only. Flashcards, quizzes and diagrams are demonstrable prototype features that test nothing about the envelope and would multiply the evaluation surface; they are excluded from measurement. Explanations are scored against document-native ground truth extracted programmatically. Of the ground-truth sources available in the corpus, this manuscript uses *per-chapter learning objectives* and *in-text cross-references*. Glossary terms are present in the corpus but are typeset as an undelimited run-on list, and recovering term boundaries reliably requires font-level information we defer to the full study; they are therefore *not* used here. We state this because a restriction disclosed is a scope decision and a restriction implied is not. **[TODO: State the exact matching function and threshold once the scorer is final, and report raw counts alongside it so the number survives a reviewer who disagrees with the threshold.]**

Dependency-ordering score. Scored independently of task quality, as forward-reference violations: the number of times a generated sequence places a concept before something the document itself declares as its prerequisite, counted for book order and for Master Plan order.

Variance. Every reported number is the result of three independent repeated runs, with spread reported alongside the central value. We report repeats rather than fixed random seeds deliberately: the inference endpoints used here do not expose deterministic decoding, so a nominal seed would name a property the system does not have. Where variance swamps an effect, that is reported as the finding.

TABLE IV
MARD (TWO-PASS) AGAINST VANILLA RLM ON THE PRIMARY DOCUMENT. THREE REPEATED RUNS; SPREAD IN PARENTHESES. BOTH ARMS AT MATCHED RECURSION DEPTH WITH IDENTICAL STRUCTURAL CONTROLS

System	Task score	Tokens	Cost
B1, vanilla RLM	–	–	–
A1, envelope removed	–	–	–
MARD (two-pass)	–	–	–
Δ (MARD − B1)	–	–	–
Δ (MARD − A1)	–	–	–

4. RESULTS AND DISCUSSION

4.1. Isolating the envelope

Table IV reports MARD in its two-pass configuration against vanilla RLM on the primary document, at matched recursion depth and under identical structural controls.

4.2. Variance across repeats

[TODO: Report the three baseline repeats individually. Do not average them: the spread in concept count, cost and wall-clock is itself the result. State explicitly that no repeat was excluded.]

4.3. Groundedness of generated content

The most consequential observation in this study was not a score. In one repeat of the baseline, the system completed without error, reported success, and produced a coherent study guide in which a substantial fraction of the explanations had been generated *without the document in context at all*.

The mechanism is worth stating precisely, because it is a property of the method rather than of our harness. In vanilla

TABLE V
GROUNDEDNESS BY SYSTEM AND REPEAT. RATE IS GROUNDED CONCEPTS AS A FRACTION OF CONCEPTS IN THE FINAL ARTEFACT

System	Rep.	Concepts	Grounded	Regen.
B1, vanilla RLM	1	–	–	–
B1, vanilla RLM	2	–	–	–
B1, vanilla RLM	3	–	–	–
MARD	1	–	–	–
MARD	2	–	–	–
MARD	3	–	–	–

RLM the root model authors its own exploration code. In this run that code mismapped a concept onto an unrelated passage; the resulting generation failed to parse and was discarded; and the retry proceeded with an empty source field, producing an explanation from the model's parametric knowledge. Nothing in the returned artefact, the exit status, or the aggregate token accounting distinguishes that explanation from a grounded one. The failure is visible only by walking the execution trajectory.

We therefore instrument groundedness directly. For each concept in a final artefact we resolve the generating call and classify it as *grounded* (non-empty source text), *ungrounded* (empty or absent source), or *regenerated* (at least one prior attempt discarded). This instrumentation was written *after* the behaviour was found by manual trajectory inspection, and we report it as such.

[TODO: Report the rate for both arms. Measure MARD's rate and report it whatever it is — claiming immunity without measuring is the same overclaim inverted. Any count of ungrounded concepts must come from the detector, not from an intermediate counter observed during the run.]

MARD's architecture bears on this structurally rather than incidentally. Builder inputs are constructed from the plan's recorded source span rather than matched at generation time; the plan is schema-validated at the tier boundary and a malformed plan fails loudly rather than dispatching many subtly incorrect builders; and the join validates non-empty output per plan step, raising rather than returning an artefact that is short by a section while satisfying every identity check.

4.4. Ablation

The frozen ablation set comprises four manipulations: A1, the envelope removed, holding MARD's architecture fixed; A2, the plan withheld from the Swarm, in which the Master Plan is still computed and still determines join order but builders receive no directive; A3, reordering disabled, joining in book order; and A4, a sweep over recursion depth reported as a curve rather than a table row. This manuscript reports A1 and A2; A3 and A4 appear in the full study.

A2 is reported alongside A1 because the two separate effects that a single comparison collapses: whether *exploration* benefits from structure-awareness, and whether *dispatch* does.

TABLE VI
ABLATION A2 — PLAN WITHHELD FROM THE SWARM

Configuration	Task score	Tokens
MARD (full)	–	–
A2 (plan withheld)	–	–

TABLE VII
NEGATIVE CONTROL. MARD AND B1 ON THE STRUCTURE-ABLATED PRIMARY DOCUMENT, THREE REPEATS EACH. A NULL DIFFERENCE IS THE PREDICTED RESULT

System	Task score	Tokens
B1, vanilla RLM	–	–
MARD (two-pass)	–	–
Δ	–	–

4.5. Negative control: structure ablated

On the structure-ablated variant, skeleton extraction recovers nothing and the envelope renders empty, so MARD's per-call input is by construction what vanilla RLM's already was. The prediction is therefore a *null difference*, and a null here is the claimed outcome rather than a failure to find an effect.

[TODO: Report the delta and its spread. If MARD is measurably *worse* here, that is the envelope's overhead with nothing to orient against, and it is a result worth stating rather than absorbing into noise.]

4.6. The structure-dependence boundary

4.7. Structure as a faithfulness mechanism

[TODO: Written against Table V. The argument: a typed, validated plan does not merely order the output, it constrains what each generation step is permitted to see. Where the baseline can fall back to parametric knowledge silently, the ablated pipeline either supplies the recorded span or fails. If MARD's own groundedness rate is below one, report and explain that instead.]

4.8. Threats to validity

4.8.1. The comparison rests on one mechanism: If the envelope ceases to propagate downward, MARD is vanilla RLM and every number in Table IV measures nothing—while every component still executes and no test fails. This mechanism is therefore guarded by dedicated tests asserting that a later chapter's prompt contains an earlier chapter's findings, and that a child envelope carries its parent's findings. We note that those tests establish the envelope is *present* in the prompt, not that the model *used* it; ablation A1 is the first direct test of influence.

4.8.2. No repeat was excluded: One baseline repeat exhibited the grounding failure described in Section 4.3, and produced markedly different output volume, cost and latency from the other two. It is reported. A run is treated as re-runnable only when the *protocol* failed to execute—a harness fault, a provider fault, an unverified corpus. A run in which

the system under test behaved badly is a result. Excluding the repeat in which the baseline performed worst would be indistinguishable from tuning toward a positive outcome.

4.8.3. Structure leakage: Section 3.3.3 describes the two routes by which document structure could reach a model without being derived. We control both and apply the controls to both arms. We cannot rule out a third route we have not thought of.

4.8.4. Single primary document: Manuscript-scale evidence from one document cannot establish that an effect is not document-specific. This is why variance across repeats, a negative control, and a pre-stated boundary carry the argument here rather than breadth, and why the full study expands the corpus.

4.8.5. Provider dependence: Results are obtained against a hosted inference endpoint whose decoding is not deterministic and whose behaviour may change. We mitigate by reporting repeats with spread and by recording a full configuration snapshot per run.

5. Limitations and Future Work

5.1. Limitations

The system evaluated here is two-pass: the targeted deep-dive pass is specified but not executed, for the reasons given in Section 3.1.3. Evaluated output is restricted to explanations, and the task score is computed against learning-objective coverage and in-text cross-references only, with the glossary source deferred (Section 3.3).

Our vanilla-RLM baseline is our own implementation of the published method rather than the original authors' code, adopted so that both arms share one implementation and the comparison isolates the envelope (Section 3.2). Runs are additionally executed under bounded sub-call concurrency and input size imposed by our provider rate-limit tier; these bounds apply to both arms, but they place the absolute scores below what an unthrottled configuration would produce, and we therefore treat the *difference* between arms as the reportable quantity and the absolute values as configuration-specific.

The flat-context control is a structure-ablated variant of our own primary document rather than an external benchmark. That choice buys a clean manipulation—content held constant, structure removed—at the cost of not demonstrating degeneration on an independently constructed flat corpus, which the full study supplies.

Whether improved structural ordering translates into learning gain is not addressed—that requires a controlled study with human participants, which we scope as future work and do not speculate about here.

5.2. Future Work

Four extensions are already scoped rather than speculative, and each is named elsewhere in this manuscript as deferred rather than abandoned. First, executing Pass 2, the targeted deep dive specified in Section 3.1.3 but not run here, once Pass 1's reliability has been evidenced over more than a single freeze window. Second, ablations A3 and A4—reordering disabled, and the sweep over recursion depth—which the full study reports and this manuscript does not, together with the external flat-context benchmark subset that complements the structure-ablated control used here (Section 4.5); its instance count places it outside this manuscript's wall-clock budget rather than outside its scope. Third, expanding the corpus beyond a single primary document, together with the dependency-ordering case study on *Linear Algebra Done Right*, whose absent learning objectives rule it out as a primary document but not as an ordering target. Fourth, and outside this project's scope, a controlled study with human participants would be required to establish whether improved structural ordering produces learning gain; we state this as an open question and do not speculate about its outcome.

Recovering glossary terms as a fourth ground-truth source, which requires font-level extraction to segment an undelimited run-on list, is a fifth deferred item rather than an abandoned one.

A sixth concerns presentation rather than measurement. The Master Plan's concept graph is currently evaluated as a scored artefact rather than rendered as one, and Fig. 1 illustrates what changes when it is. Deployed products that generate document-derived mind maps render containment: the structure a document already declares through its headings. A prerequisite graph affords a different interaction—entering the material at a concept and walking backwards through what it depends on, rather than downwards through what contains it. Whether that serves a learner better than a hierarchical view is an open question, and answering it requires the controlled study scoped above rather than an appeal to either rendering's appearance.

6. Conclusion

References

[1] A. L. Zhang, T. Kraska, and O. Khattab, "Recursive language models," 2025.

[2] K. Alizadeh, P. Shojaee, M. Cho, and M. Farajtabar, "SRLM: Scaling recursive language models with multi-pass attention," 2026.

[3] N. F. Liu, K. Lin, J. Hewitt, A. Paranjape, M. Bevilacqua, F. Petroni, and P. Liang, "Lost in the middle: How language models use long contexts," *Transactions of the Association for Computational Linguistics*, 2024.

[4] H. Jiang, Q. Wu, C.-Y. Lin, Y. Yang, and L. Qiu, "LLMLingua: Compressing prompts for accelerated inference of large language models," in *Proc. EMNLP*, 2023.

[5] Z. Pan, Q. Wu, H. Jiang, M. Xia, X. Luo, and J. Zhang, "LLMLingua-2: Data distillation for efficient and faithful task-agnostic prompt compression," in *Findings of the ACL*, 2024.

[6] Y. Li, B. Dong, F. Guerin, and C. Lin, "Compressing context to enhance inference efficiency of large language models," in *Proc. EMNLP*, 2023.

[7] Z. Li, Y. Liu, Y. Su, and N. Collier, "Prompt compression for large language models: A survey," in *Proc. NAACL*, 2025.

[8] L. Ou and M. Lapata, "Context-aware hierarchical merging for long document summarization," in *Findings of the ACL*, 2025.

[9] H. Kim and B. H. Kim, "NexusSum: Hierarchical LLM agents for long-form narrative summarization," in *Proc. ACL*, 2025.

[10] I. Ong, A. Almahairi, V. Wu, W.-L. Chiang, T. Wu, J. E. Gonzalez, M. W. Kadous, and I. Stoica, "RouteLLM: Learning to route LLMs with preference data," 2024.

[11] Y. Moslem and J. D. Kelleher, "Dynamic model routing and cascading for efficient LLM inference: A survey," 2026.

[12] Z. Dong, H. Sharma, E. O'Toole, J. P. Champati, and K. Wu, "Pay for hints, not answers: LLM shepherding for cost-efficient inference," 2026.

[13] I. Li, V. Yan, T. Li, R. Qu, and D. Radev, "Unsupervised cross-domain prerequisite chain learning using variational graph autoencoders," in *Proc. ACL-IJCNLP*, 2021, pp. 1005–1011.
[14] J. Sun, Y. He, Y. Xu, J. Sun, and G. Sun, "A learning-path based supervised method for concept prerequisite relations extraction," in *Proc. ACM CIKM*, 2024, pp. 2168–2177.
[15] "Prerequisite relation learning: A survey and outlook," *ACM Computing Surveys*, vol. 57, no. 11, 2025.
[16] S. Wu, "LLM agents for education: Advances and applications," 2025.
[17] E. Tufino, "NotebookLM as a Socratic physics tutor: Design and preliminary observations of a RAG-based tool," 2025.